



Finite Difference Method and Poisson equation

Hervé Bulou

► To cite this version:

Hervé Bulou. Finite Difference Method and Poisson equation. Doctoral. TPMN, University of Strasbourg, France. 2025, pp.22. hal-05233941

HAL Id: hal-05233941

<https://hal.science/hal-05233941v1>

Submitted on 1 Sep 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons Attribution - NonCommercial 4.0 International License

Finite Difference Method & Poisson equation

Hervé Bulou*

Université de Strasbourg, CNRS, IPCMS, UMR 7504, F-67000 Strasbourg, France,

(Dated: September 1, 2025)

PACS numbers:

I. INTRODUCTION

Density Functional Theory (DFT) in the Kohn-Sham formulation (KS-DFT) is one of the most efficient way to get the electron distribution, $\rho(\mathbf{r}, t)$, of any quantum system in the ground state[1–3]. An important part of the KS-DFT calculation is to compute the electrostatic potential (also named Hartree potential), $\phi(\mathbf{r}, t)$, associated to the electron distribution. The most well-known way to compute it is to resort to the integral form of Coulomb’s law:

$$\phi(\mathbf{r}) = \frac{1}{4\pi\epsilon_0} \int \int \int \frac{\rho(\mathbf{r}')}{\|\mathbf{r} - \mathbf{r}'\|} d\mathbf{r}', \quad (\text{I.1})$$

where $\epsilon_0 = 8.854 \times 10^{-12}$ F/m is the permittivity of free space. However, numerical implementation of equation (I.1),

$$\phi(\mathbf{r}_i) = \frac{1}{4\pi\epsilon_0} \sum_{j=1}^N \frac{\rho(\mathbf{r}_j)}{\|\mathbf{r}_i - \mathbf{r}_j\|}, \quad (\text{I.2})$$

is computationally expensive, since it needs N^2 operations to compute ϕ at each point in the space by direct summation. A more efficient way to proceed is to solve Poisson’s partial differential equation[4, 5]:

$$\nabla^2 \phi(\mathbf{r}) = -\frac{\rho(\mathbf{r})}{\epsilon_0}. \quad (\text{I.3})$$

Poisson’s equation first appearance was in electrostatics, when the French mathematician Siméon-Denis Poisson generalized Laplace’s equation $\nabla^2 V = 0$ to nonvacuum regions[6].

While Poisson’s equation allows to speed up the computation of the Coulombian interactions, some points, at the boundaries of the systems, needs to be computed by resorting to direct summation.

L’interaction Coulombienne n’étant pas une interaction quantique mais une interaction classique, un nombre important de m Several ways were proposed to speed up direct summation

- particle-mesh methods[7–10],
- multilevel approximation schemes of Brandt and Lubrecht[11, 12],
- “wavelet”-based scheme[13],
- multipole expansions[19–22] which was developed independently in
 - the astrophysics[14, 15],
 - fluid dynamics[16],
 - number theory[17],
 - potential theory[18, 19],

Le principe de la méthode FMM est de traiter différemment les interactions entre électrons situés loin l’un de l’autre (interaction à longue portée) et les interactions entre électrons voisins (interaction à courte portée).

La plupart de ces méthodes reposent sur des “coarse graining scheme” dans lesquels les interactions entre les particules sont traitées différemment selon la distance (ou l’intervalle de temps en dynamique moléculaire). Les

*Electronic address: herve.bulou@ipcms.unistra.fr

“spatial coarse graining methods” reposent sur le fait que des distributions de charges éloignées interagissent moins fortement entre elles que lorsque’elles sont proches. Des méthodes ont ainsi été développées

J. V. Villumsen, Astron. J. Suppl. Ser. 71, 407 (1989).

Numerous algorithms exist to solve the Poisson equation:

- direct[23, 24]
- iterative solvers[25]
- Fourier spectral methods[26, 27]
- Green’s function method
- Fast Multipole Method[19]
- Multigrid Method

In these practicals, we will consider two methods for solving the Poisson equation for a 2D system: the Finite Difference Method (FDM) and the Finite Element Method (FEM).

II. THE FINITE DIFFERENCE METHOD - 1D CASE

A. Method

The Finite Difference Method (FDM) consists in replacing the continuous derivative Laplacian operator, ∇^2 , involved in the Poisson equation, with finite-difference approximations, and then numerically solving the resultant system.

The first step is to define a *mesh*, a set spatial points at which the quantities involved in the system (electrostatic potential, charge distribution, ...) are sampled.

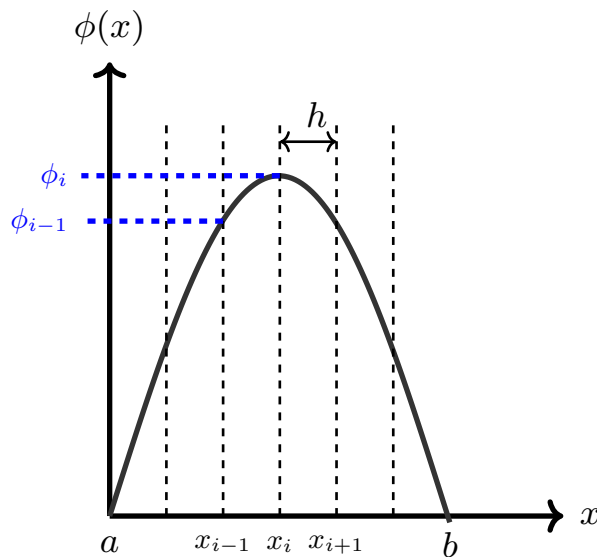


Figure 1: Finite difference mesh in 1D.

Figure 1 displays an example of mesh used in the FDM. Let h the step between each nearest neighbors nodes of the mesh. Each nodes of the mesh can be defined by a coordinates $x_i = ih$ where i is integers. In practice, i and j are used as indices for the matrix representing the quantities of interest (electrostatic potential, charge distribution, ...): $\phi(x_i, y_i) \equiv \phi(i, j)$.

In practice, i is used as index for the matrix representing the quantities of interest (electrostatic potential, charge distribution, ...): $\phi(x_i) \equiv \phi(i)$.

The next step is to approximate the Laplacian operator, ∇^2 , acting on the function ϕ , as a weighted sums of function values. To compute the weighted coefficients, one starts from the Taylor expansions of $\phi(x)$ [29]:

$$\phi(x+h) = \phi(x) + h \frac{\partial \phi(x)}{\partial x} + \frac{h^2}{2} \frac{\partial^2 \phi(x)}{\partial x^2} + \mathcal{O}(h^3), \quad (\text{II.1})$$

$$\phi(x) = \phi(x), \quad (\text{II.2})$$

$$\phi(x-h) = \phi(x) + h \frac{\partial \phi(x)}{\partial x} - \frac{h^2}{2} \frac{\partial^2 \phi(x,y)}{\partial x^2} + \mathcal{O}(h^3). \quad (\text{II.3})$$

Forming the sum

$$a\phi(x+h) + b\phi(x) + c\phi(x-h) = (a+b+c)\phi(x) + (a-c)h \frac{\partial \phi(x)}{\partial x} + (a+c) \frac{h^2}{2} \frac{\partial^2 \phi(x)}{\partial x^2} + \mathcal{O}(h^3). \quad (\text{II.4})$$

From equation (II.4), we observe that an expression of the second derivative of ϕ can be obtained by solving the linear system:

$$a+b+c = 0 \quad (\text{II.5})$$

$$a-c = 0 \quad (\text{II.6})$$

$$a+c = 1 \quad (\text{II.7})$$

$$(\text{II.8})$$

leading to $a = c = 1$ and $b = -2$:

$$\frac{\partial^2 \phi(i)}{\partial x^2} \simeq \frac{\phi(i-1) - 2\phi(i) + \phi(i+1)}{h^2}. \quad (\text{II.9})$$

The Laplacian operator, in 1D, $\nabla^2 \phi(i) = \frac{\partial^2 \phi(i)}{\partial x^2}$, is therefore approximated by

$$\nabla^2 \phi(i) \simeq \frac{\phi(i-1) - 2\phi(i) + \phi(i+1)}{h^2} = -\frac{\rho(i)}{\varepsilon_0}, \quad (\text{II.10})$$

$$\frac{1}{h^2} (\phi(i-1) - 2\phi(i) + \phi(i+1)) = -\frac{\rho(i)}{\varepsilon_0}. \quad (\text{II.11})$$

One observes from Eq. (II.11) that the discretization of the Poisson equation links the value of the electrostatic potential at any point i with its neighbors and the charge at this point. There are as many equations (II.11) as there are nodes in the mesh, one for each node.

Now considering the case displayed on Fig. 1, there is 7 points from $x_0 = a$ to $x_6 = b$. The values of the electrostatic potential at x_0 and x_6 are known and we want to compute the electrostatic potential for the points included between x_1 and x_5 . Thus we can write a set of five coupled equations from the discretization of the 1D Poisson equation:

$$\begin{aligned} (1) \Rightarrow & \frac{1}{h^2} \phi(0) - \frac{2}{h^2} \phi(1) + \frac{1}{h^2} \phi(2) = -\frac{\rho(1)}{\varepsilon_0} \\ (2) \Rightarrow & \frac{1}{h^2} \phi(1) - \frac{2}{h^2} \phi(2) + \frac{1}{h^2} \phi(3) = -\frac{\rho(2)}{\varepsilon_0} \\ (3) \Rightarrow & \frac{1}{h^2} \phi(2) - \frac{2}{h^2} \phi(3) + \frac{1}{h^2} \phi(4) = -\frac{\rho(3)}{\varepsilon_0} \\ (4) \Rightarrow & \frac{1}{h^2} \phi(3) - \frac{2}{h^2} \phi(4) + \frac{1}{h^2} \phi(5) = -\frac{\rho(4)}{\varepsilon_0} \\ (5) \Rightarrow & \frac{1}{h^2} \phi(4) - \frac{2}{h^2} \phi(5) + \frac{1}{h^2} \phi(6) = -\frac{\rho(5)}{\varepsilon_0} \end{aligned}$$

In the above equations, the red terms, $\phi(0)$ and $\phi(6)$, represent the Hartree potential at the boundaries of the computing domain. They do not belong to the computational domain and they must be provided for solving the system. By putting them on the right side of the equations, we get

$$\begin{aligned}
(1) \Rightarrow \quad & -\frac{2}{h^2}\phi(1) + \frac{1}{h^2}\phi(2) = -\frac{\rho(1)}{\varepsilon_0} - \frac{1}{h^2}\phi(0) \\
(2) \Rightarrow \quad & \frac{1}{h^2}\phi(1) - \frac{2}{h^2}\phi(2) + \frac{1}{h^2}\phi(3) = -\frac{\rho(2)}{\varepsilon_0} \\
(3) \Rightarrow \quad & \frac{1}{h^2}\phi(2) - \frac{2}{h^2}\phi(3) + \frac{1}{h^2}\phi(4) = -\frac{\rho(3)}{\varepsilon_0} \\
(4) \Rightarrow \quad & \frac{1}{h^2}\phi(3) - \frac{2}{h^2}\phi(4) + \frac{1}{h^2}\phi(5) = -\frac{\rho(4)}{\varepsilon_0} \\
(5) \Rightarrow \quad & \frac{1}{h^2}\phi(4) - \frac{2}{h^2}\phi(5) = -\frac{\rho(5)}{\varepsilon_0} - \frac{1}{h^2}\phi(6)
\end{aligned}$$

The above equations can be arranged in order to get a matrix form:

$$\frac{1}{h^2} \begin{bmatrix} -2 & 1 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 1 & -2 & 0 \\ 0 & 0 & 0 & 1 & -2 \end{bmatrix} \begin{bmatrix} \phi(1) \\ \phi(2) \\ \phi(3) \\ \phi(4) \\ \phi(5) \end{bmatrix} = -\frac{1}{\varepsilon_0} \begin{bmatrix} \rho(1) \\ \rho(2) \\ \rho(3) \\ \rho(4) \\ \rho(5) \end{bmatrix} - \frac{1}{h^2} \begin{bmatrix} \phi(0) \\ 0 \\ 0 \\ 0 \\ \phi(6) \end{bmatrix}$$

$$\mathbf{L} |\phi\rangle = -\frac{1}{\varepsilon_0} |\rho\rangle - \frac{1}{h^2} |b\rangle. \quad (\text{II.12})$$

Except $|\phi\rangle$, all is known in the above equation. The Hartree potential can be determined by using

- either the LAPACK subroutine **dgesv**,
- or by using a conjugate gradient method.

B. Test

We're going to consider a system with a spherical symmetry and where the electrostatic potential, $V(\mathbf{r})$, and the source, $\rho(\mathbf{r})$, depend only on r . In this case the Laplacian $\nabla^2 V(\mathbf{r})$ can be written as[30]

$$\frac{1}{r^2} \frac{\partial}{\partial r} \left(r^2 \frac{\partial V(\mathbf{r})}{\partial r} \right) + \frac{1}{r^2 \sin \theta} \frac{\partial}{\partial \theta} \left(\sin \theta \frac{\partial V(\mathbf{r})}{\partial \theta} \right) + \frac{1}{r^2 \sin^2 \theta} \frac{\partial^2 V(\mathbf{r})}{\partial \varphi^2}. \quad (\text{II.13})$$

Since $V(\mathbf{r}) = V(r)$ and $\rho(\mathbf{r}) = \rho(r)$, the Poisson's equation is (in Hartree and Bohr radius units)

$$\frac{1}{r^2} \frac{d}{dr} \left(r^2 \frac{dV(r)}{dr} \right) = -4\pi r \rho(r). \quad (\text{II.14})$$

Setting $V(r) = r^{-1}U(r)$, we get

$$\frac{d^2 U(r)}{dr^2} = -4\pi r \rho(r), \quad (\text{II.15})$$

which can be solved by resorting to Numerov method with $Q(r) = 0$ and $S(r) = -4\pi r \rho(r)$.

Figure 2(a) displays an example of the electrostatic potential stemming from an electron distribution given by $\rho(r) = \frac{\alpha}{2\pi\sigma^2} \left(3 - \frac{2r^2}{\sigma^2} \right) e^{-r^2/\sigma^2}$ (Figure 2(b)). The corresponding electrostatic potential is given by $V(r) = \alpha e^{-r^2/\sigma^2}$.

III. THE FINITE DIFFERENCE METHOD - 2D CASE

The Finite Difference Method (FDM) consists in replacing the continuous derivative Laplacian operator, ∇^2 , involved in the Poisson equation, with finite-difference approximations, and then numerically solving the resultant system.

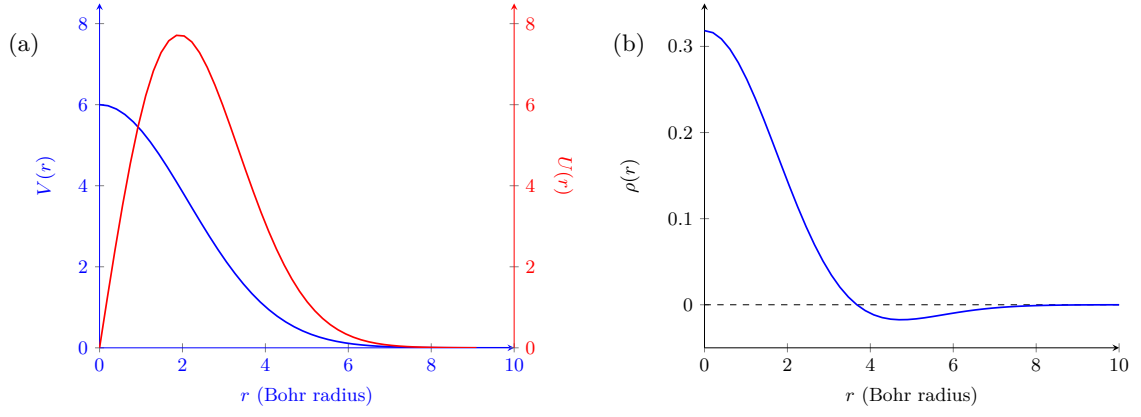


Figure 2: DD

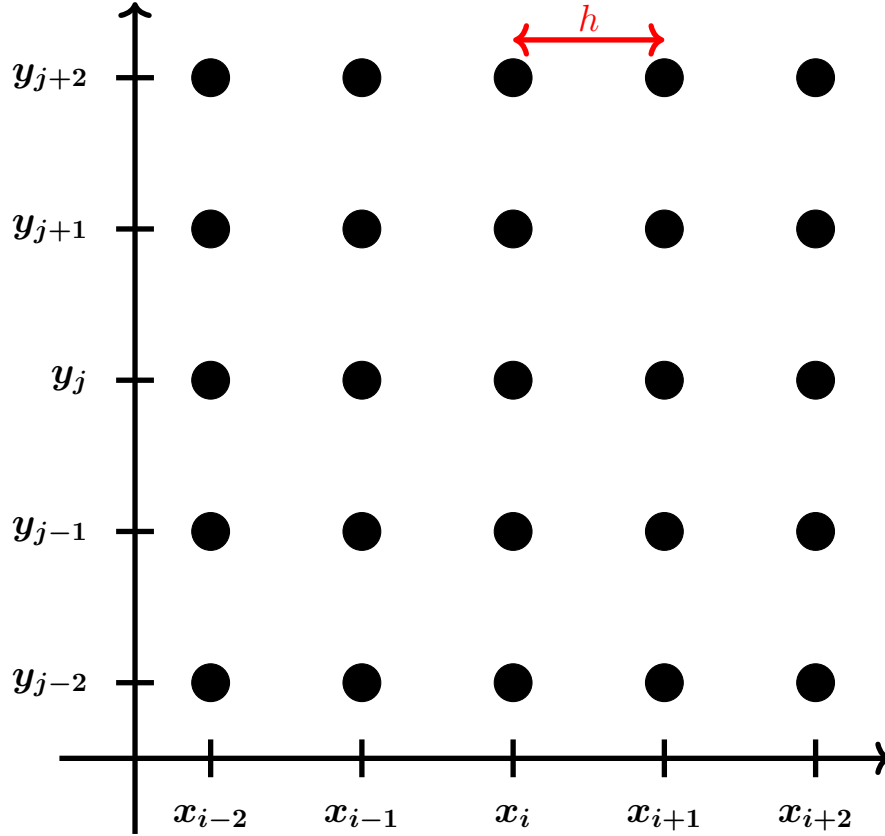


Figure 3: Finite difference grid.

The first step is to define a *mesh*, a set spatial points at which the quantities involved in the system (electrostatic potential, charge distribution, ...) are sampled. Figure 3 displays an example of mesh used in the FDM. Let h the step between each nearest neighbors nodes of the mesh. Each nodes of the mesh can be defined by a couple of coordinates $x_i = ih$ and $y_j = jh$, where i and j are integers. In practice, i and j are used as indices for the matrix representing the quantities of interest (electrostatic potential, charge distribution, ...): $\phi(x_i, y_i) \equiv \phi(i, j)$.

The next step is to approximate the Laplacian operator, ∇^2 , acting on the function ϕ , as a weighted sums of

function values. To compute the weighted coefficients, one starts from the Taylor expansions of $\phi(x)$ [29]:

$$\phi(x+h, y) = \phi(x, y) + h \frac{\partial \phi(x, y)}{\partial x} + \frac{h^2}{2} \frac{\partial^2 \phi(x, y)}{\partial x^2} + \mathcal{O}(h^3), \quad (\text{III.1})$$

$$\phi(x, y) = \phi(x, y), \quad (\text{III.2})$$

$$\phi(x-h, y) = \phi(x, y) + h \frac{\partial \phi(x, y)}{\partial x} - \frac{h^2}{2} \frac{\partial^2 \phi(x, y)}{\partial x^2} + \mathcal{O}(h^3). \quad (\text{III.3})$$

Forming the sum

$$a\phi(x+h, y) + b\phi(x, y) + c\phi(x-h, y) = (a+b+c)\phi(x, y) + (a-c)h \frac{\partial \phi(x, y)}{\partial x} + (a+c) \frac{h^2}{2} \frac{\partial^2 \phi(x, y)}{\partial x^2} + \mathcal{O}(h^3). \quad (\text{III.4})$$

From equation (III.4), we observe that an expression of the second derivative of ϕ can be obtained by solving the linear system:

$$a + b + c = 0 \quad (\text{III.5})$$

$$a - c = 0 \quad (\text{III.6})$$

$$a + c = 1 \quad (\text{III.7})$$

$$(\text{III.8})$$

leading to $a = c = 1$ and $b = -2$:

$$\frac{\partial^2 \phi(i, j)}{\partial x^2} \simeq \frac{\phi(i-1, j) - 2\phi(i, j) + \phi(i+1, j)}{h^2}. \quad (\text{III.9})$$

One can derive a similar expression for y :

$$\frac{\partial^2 \phi(i, j)}{\partial y^2} \simeq \frac{\phi(i, j-1) - 2\phi(i, j) + \phi(i, j+1)}{h^2}, \quad (\text{III.10})$$

The Laplacian operator, in 2D, $\nabla^2 \phi(i, j) = \frac{\partial^2 \phi(i, j)}{\partial x^2} + \frac{\partial^2 \phi(i, j)}{\partial y^2}$, is therefore approximated by

$$\nabla^2 \phi(i, j) \simeq \frac{\phi(i-1, j) - 2\phi(i, j) + \phi(i+1, j)}{h^2} + \frac{\phi(i, j-1) - 2\phi(i, j) + \phi(i, j+1)}{h^2} = -\frac{\rho(i, j)}{\varepsilon_0}, \quad (\text{III.11})$$

$$\frac{1}{h^2} (\phi(i-1, j) + \phi(i, j-1) - 4\phi(i, j) + \phi(i+1, j) + \phi(i, j+1)) = -\frac{\rho(i, j)}{\varepsilon_0}. \quad (\text{III.12})$$

One observes from Eq. (III.12) that the discretization of the Poisson equation links the value of the electrostatic potential at any point (i, j) with its neighbors and the charge at this point. There are as many equations (III.12) as there are nodes in the mesh, one for each node.

Figure 5 displays an example of a $N_x (= 4) \times N_y (= 3)$ grid used for the FDM. Solving the Poisson equation within the Five-Point star approximation for the Laplacian needs to consider a set of $N_x \times N_y = 12$ coupled equations

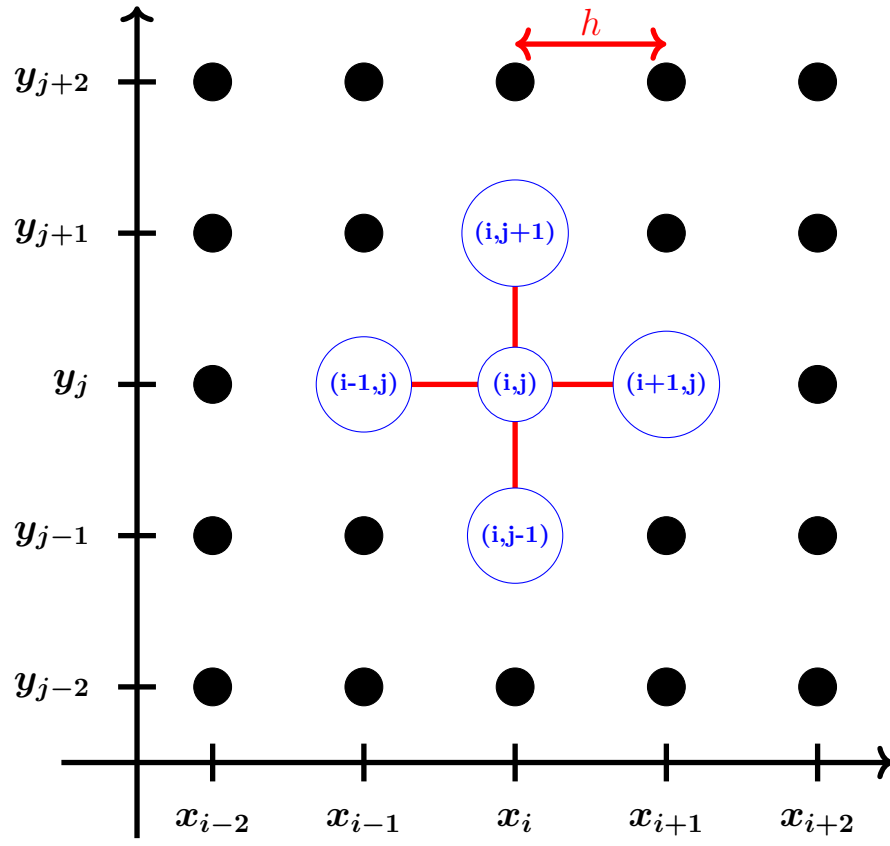


Figure 4: Computational grid used for the FDM: the Five-Point star.

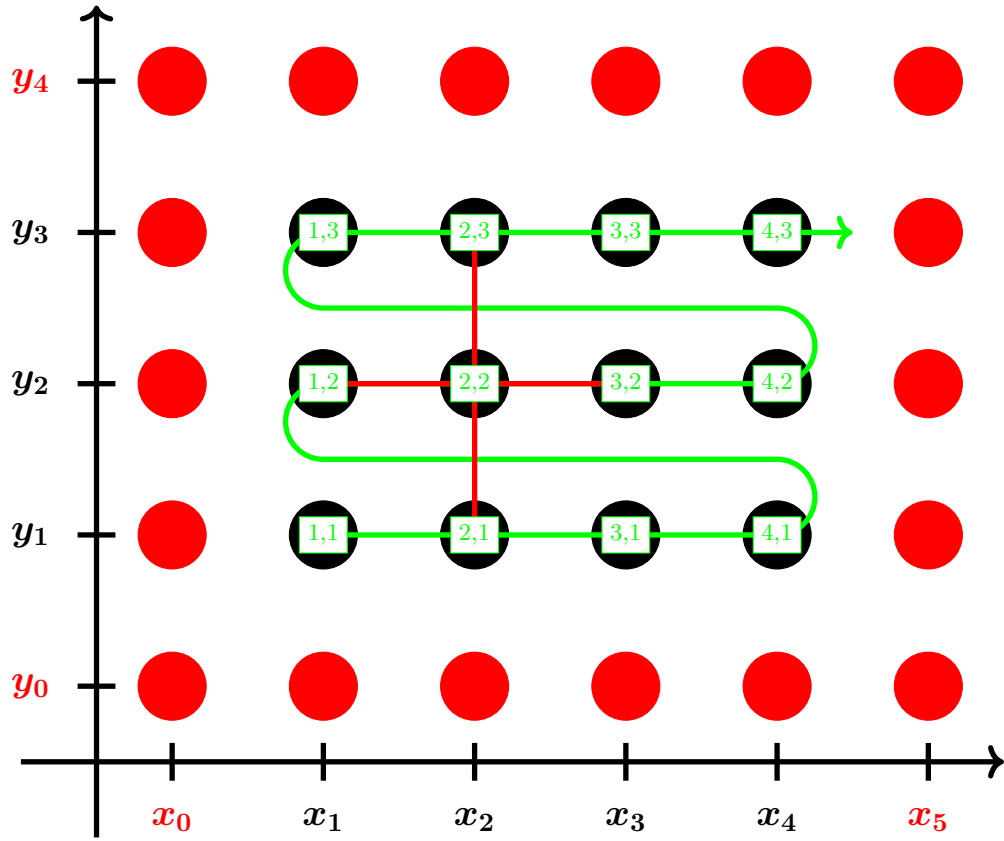


Figure 5: Example of $N_x(=4) \times N_y(=3)$ grid used for the FDM. the Black nodes represent the points where the potential energy is computed, the red ones the boundaries of the computing domain.

$$\begin{aligned}
(1, 1) &\Rightarrow \frac{1}{h^2}\phi(0, 1) + \frac{1}{h^2}\phi(1, 0) - \frac{4}{h^2}\phi(1, 1) + \frac{1}{h^2}\phi(2, 1) + \frac{1}{h^2}\phi(1, 2) = -\frac{\rho(1, 1)}{\varepsilon_0} \\
(2, 1) &\Rightarrow \frac{1}{h^2}\phi(1, 1) + \frac{1}{h^2}\phi(2, 0) - \frac{4}{h^2}\phi(2, 1) + \frac{1}{h^2}\phi(3, 1) + \frac{1}{h^2}\phi(2, 2) = -\frac{\rho(2, 1)}{\varepsilon_0} \\
(3, 1) &\Rightarrow \frac{1}{h^2}\phi(2, 1) + \frac{1}{h^2}\phi(3, 0) - \frac{4}{h^2}\phi(3, 1) + \frac{1}{h^2}\phi(4, 1) + \frac{1}{h^2}\phi(3, 2) = -\frac{\rho(3, 1)}{\varepsilon_0} \\
(4, 1) &\Rightarrow \frac{1}{h^2}\phi(3, 1) + \frac{1}{h^2}\phi(4, 0) - \frac{4}{h^2}\phi(4, 1) + \frac{1}{h^2}\phi(5, 1) + \frac{1}{h^2}\phi(4, 2) = -\frac{\rho(4, 1)}{\varepsilon_0} \\
(1, 2) &\Rightarrow \frac{1}{h^2}\phi(0, 2) + \frac{1}{h^2}\phi(1, 1) - \frac{4}{h^2}\phi(1, 2) + \frac{1}{h^2}\phi(2, 2) + \frac{1}{h^2}\phi(1, 3) = -\frac{\rho(1, 2)}{\varepsilon_0} \\
(2, 2) &\Rightarrow \frac{1}{h^2}\phi(1, 2) + \frac{1}{h^2}\phi(2, 1) - \frac{4}{h^2}\phi(2, 2) + \frac{1}{h^2}\phi(3, 2) + \frac{1}{h^2}\phi(2, 3) = -\frac{\rho(2, 2)}{\varepsilon_0} \\
(3, 2) &\Rightarrow \frac{1}{h^2}\phi(2, 2) + \frac{1}{h^2}\phi(3, 1) - \frac{4}{h^2}\phi(3, 2) + \frac{1}{h^2}\phi(4, 2) + \frac{1}{h^2}\phi(3, 3) = -\frac{\rho(3, 2)}{\varepsilon_0} \\
(4, 2) &\Rightarrow \frac{1}{h^2}\phi(3, 2) + \frac{1}{h^2}\phi(4, 1) - \frac{4}{h^2}\phi(4, 2) + \frac{1}{h^2}\phi(5, 2) + \frac{1}{h^2}\phi(4, 3) = -\frac{\rho(4, 2)}{\varepsilon_0} \\
(1, 3) &\Rightarrow \frac{1}{h^2}\phi(0, 3) + \frac{1}{h^2}\phi(1, 2) - \frac{4}{h^2}\phi(1, 3) + \frac{1}{h^2}\phi(2, 3) + \frac{1}{h^2}\phi(1, 4) = -\frac{\rho(1, 3)}{\varepsilon_0} \\
(2, 3) &\Rightarrow \frac{1}{h^2}\phi(1, 3) + \frac{1}{h^2}\phi(2, 2) - \frac{4}{h^2}\phi(2, 3) + \frac{1}{h^2}\phi(3, 3) + \frac{1}{h^2}\phi(2, 4) = -\frac{\rho(2, 3)}{\varepsilon_0} \\
(3, 3) &\Rightarrow \frac{1}{h^2}\phi(2, 3) + \frac{1}{h^2}\phi(3, 2) - \frac{4}{h^2}\phi(3, 3) + \frac{1}{h^2}\phi(4, 3) + \frac{1}{h^2}\phi(3, 4) = -\frac{\rho(3, 3)}{\varepsilon_0} \\
(4, 3) &\Rightarrow \frac{1}{h^2}\phi(3, 3) + \frac{1}{h^2}\phi(4, 2) - \frac{4}{h^2}\phi(4, 3) + \frac{1}{h^2}\phi(5, 3) + \frac{1}{h^2}\phi(4, 4) = -\frac{\rho(4, 3)}{\varepsilon_0}
\end{aligned}$$

In the above equations, the red terms represent the Hartree potential at the boundaries of the computing domain. They do not belong to the computational domain and they must be provided for solving the system.

$$\begin{aligned}
-\frac{4}{h^2}\phi(1,1) + \frac{1}{h^2}\phi(2,1) + \frac{1}{h^2}\phi(1,2) &= -\frac{\rho(1,1)}{\varepsilon_0} - \frac{1}{h^2}\phi(0,1) - \frac{1}{h^2}\phi(1,0) \\
\frac{1}{h^2}\phi(1,1) - \frac{4}{h^2}\phi(2,1) + \frac{1}{h^2}\phi(3,1) + \frac{1}{h^2}\phi(2,2) &= -\frac{\rho(2,1)}{\varepsilon_0} - \frac{1}{h^2}\phi(2,0) \\
\frac{1}{h^2}\phi(2,1) - \frac{4}{h^2}\phi(3,1) + \frac{1}{h^2}\phi(4,1) + \frac{1}{h^2}\phi(3,2) &= -\frac{\rho(3,1)}{\varepsilon_0} - \frac{1}{h^2}\phi(3,0) \\
\frac{1}{h^2}\phi(3,1) - \frac{4}{h^2}\phi(4,1) + \frac{1}{h^2}\phi(4,2) &= -\frac{\rho(4,1)}{\varepsilon_0} - \frac{1}{h^2}\phi(4,0) - \frac{1}{h^2}\phi(5,1) \\
\frac{1}{h^2}\phi(1,1) - \frac{4}{h^2}\phi(1,2) + \frac{1}{h^2}\phi(2,2) + \frac{1}{h^2}\phi(1,3) &= -\frac{\rho(1,2)}{\varepsilon_0} - \frac{1}{h^2}\phi(0,2) \\
\frac{1}{h^2}\phi(1,2) + \frac{1}{h^2}\phi(2,1) - \frac{4}{h^2}\phi(2,2) + \frac{1}{h^2}\phi(3,2) + \frac{1}{h^2}\phi(2,3) &= -\frac{\rho(2,2)}{\varepsilon_0} \\
\frac{1}{h^2}\phi(2,2) + \frac{1}{h^2}\phi(3,1) - \frac{4}{h^2}\phi(3,2) + \frac{1}{h^2}\phi(4,2) + \frac{1}{h^2}\phi(3,3) &= -\frac{\rho(3,2)}{\varepsilon_0} \\
\frac{1}{h^2}\phi(3,2) + \frac{1}{h^2}\phi(4,1) - \frac{4}{h^2}\phi(4,2) + \frac{1}{h^2}\phi(4,3) &= -\frac{\rho(4,2)}{\varepsilon_0} - \frac{1}{h^2}\phi(5,2) \\
\frac{1}{h^2}\phi(1,2) - \frac{4}{h^2}\phi(1,3) + \frac{1}{h^2}\phi(2,3) &= -\frac{\rho(1,3)}{\varepsilon_0} - \frac{1}{h^2}\phi(0,3) - \frac{1}{h^2}\phi(1,4) \\
\frac{1}{h^2}\phi(1,3) + \frac{1}{h^2}\phi(2,2) - \frac{4}{h^2}\phi(2,3) + \frac{1}{h^2}\phi(3,3) &= -\frac{\rho(2,3)}{\varepsilon_0} - \frac{1}{h^2}\phi(2,4) \\
\frac{1}{h^2}\phi(2,3) + \frac{1}{h^2}\phi(3,2) - \frac{4}{h^2}\phi(3,3) + \frac{1}{h^2}\phi(4,3) &= -\frac{\rho(3,3)}{\varepsilon_0} - \frac{1}{h^2}\phi(3,4) \\
\frac{1}{h^2}\phi(3,3) + \frac{1}{h^2}\phi(4,2) - \frac{4}{h^2}\phi(4,3) &= -\frac{\rho(4,3)}{\varepsilon_0} - \frac{1}{h^2}\phi(5,3) - \frac{1}{h^2}\phi(4,4)
\end{aligned}$$

By mapping the 2D distribution of Hartree potential, $\phi(i, j)$, into a vector representation using the green path on Figure 5, the above equations can be arranged in a matrix-vector form

$$\frac{1}{h^2} \begin{bmatrix} -4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & -4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -4 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & -4 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & -4 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & -4 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & -4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & -4 \end{bmatrix} \begin{bmatrix} \phi(1,1) \\ \phi(2,1) \\ \phi(3,1) \\ \phi(4,1) \\ \phi(1,2) \\ \phi(2,2) \\ \phi(3,2) \\ \phi(4,2) \\ \phi(1,3) \\ \phi(2,3) \\ \phi(3,3) \\ \phi(4,3) \end{bmatrix} = -\frac{1}{\varepsilon_0} \begin{bmatrix} \rho(1,1) \\ \rho(2,1) \\ \rho(3,1) \\ \rho(4,1) \\ \rho(1,2) \\ \rho(2,2) \\ \rho(3,2) \\ \rho(4,2) \\ \rho(1,3) \\ \rho(2,3) \\ \rho(3,3) \\ \rho(4,3) \end{bmatrix} - \frac{1}{h^2} \begin{bmatrix} \phi(0,1) + \phi(1,0) \\ \phi(2,0) \\ \phi(3,0) \\ \phi(4,0) + \phi(5,1) \\ \phi(0,2) \\ 0 \\ 0 \\ \phi(5,2) \\ \phi(0,3) + \phi(1,4) \\ \phi(2,4) \\ \phi(3,4) \\ \phi(5,3) + \phi(4,4) \end{bmatrix}$$

$$\mathbf{L} |\phi\rangle = -\frac{1}{\varepsilon_0} |\rho\rangle - \frac{1}{h^2} |b\rangle. \quad (\text{III.13})$$

Except $|\phi\rangle$, all is known in the above equation. The Hartree potential can be determined by using

- either the LAPACK subroutine **dgesv**,
- or by using a conjugate gradient method.

IV. FINITE ELEMENT METHOD (FEM) VS FDM

- the FDM is generally solved within a fixed, rectangular mesh while the FEM uses a flexible, triangular mesh.

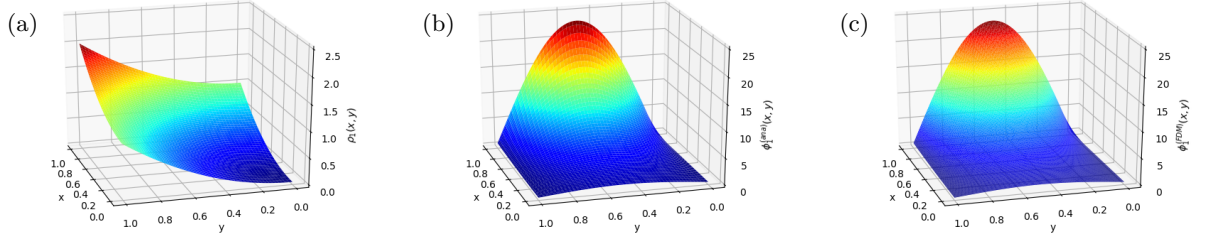


Figure 6: Example 1: (a) source and electrostatic potential computed (b) analytically and (c) by using the finite difference method (100×100 grid).

- The FEM is more complex to implement compared to FDM
- ...

V. EXAMPLES OF POISSON EQUATIONS IN TWO-DIMENSIONS

We consider the 2D Poisson equation

$$\Delta\phi(x, y) = \rho(x, y) \quad (\text{V.1})$$

in the unit square $(x, y) \in \Omega = [0, 1] \times [0, 1]$.

A. Example 1

It can be proved that the source $\rho(x, y) = x^2 + y^2$ (Fig. 6(a)) is at the origin of an electrostatic potential given by the expression

$$\phi(x, y) = e^{\pi x} \sin(\pi y) + \frac{1}{2} (xy)^2. \quad (\text{V.2})$$

Using the finite difference method and equation (V.2) to get the boundary conditions along the $x = 0$ and $y = 0$ directions, solve the Poisson equation (I.3).

B. Example 2

We consider the source $\rho(x, y) = -20 \cos(3\pi x) \sin(2\pi y)$ (Fig. 7(a)).

Using the finite difference method with boundary conditions $\phi(0, y) = y^2$, $\phi(1, y) = 1$, $\phi(x, 0) = x^3$, and $\phi(x, 1) = 1$, solve the Poisson equation (I.3).

C. Example 3

We consider the source $\rho(r) = \alpha \left(\frac{r^2}{\sigma^2} - 1 \right) e^{-r^2/\sigma^2}$ with $r = \sqrt{(x - x_0)^2 + (y - y_0)^2}$ (Fig. 8(c)).

Using

- the finite difference method,
 - $\alpha = 10$, $(x_0, y_0) = (\frac{1}{2}, \frac{1}{2})$, $\sigma = 0.2$,
 - and by computing the the boundary conditions along the $x = 0$ and $y = 0$ directions $\phi(r) = 0.25\alpha\sigma^2 e^{-r^2/\sigma^2}$,
- solve the Poisson equation (I.3).

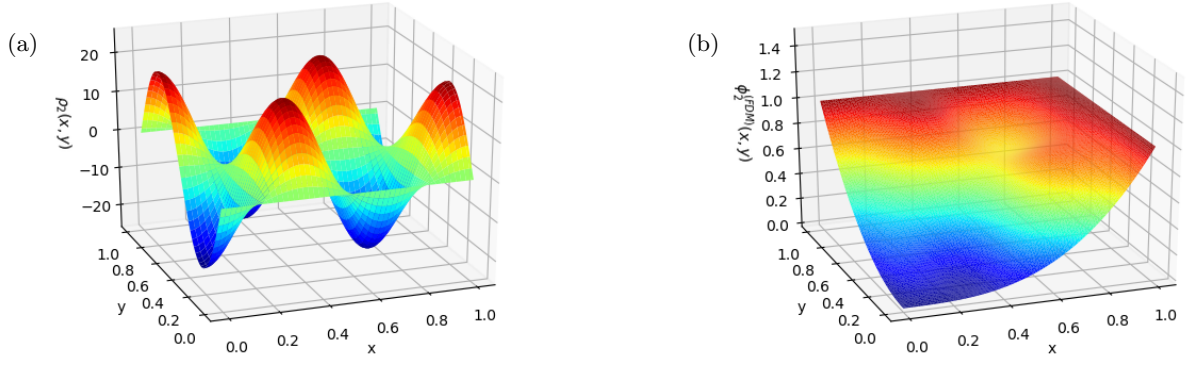


Figure 7: Example 2: (a) source and (b) electrostatic potential computed by using the finite difference method (100×100 grid).

D. Example 4

We consider the source $\rho(r) = \sum_{i=1}^2 \alpha_i \left(\frac{r^2}{\sigma_i^2} - 1 \right) e^{-r^2/\sigma_i^2}$ with $r = \sqrt{(x - x_0)^2 + (y - y_0)^2}$ ((Fig. 9(c)). Using

- the finite difference method,
- $\alpha_1 = 10$, $\sigma_1 = 0.2$, $\alpha_2 = 60$, $\sigma_2 = 0.1$, $(x_0, y_0) = (\frac{1}{2}, \frac{1}{2})$,
- and by computing the the boundary conditions along the $x = 0$ and $y = 0$ directions $\phi(r) = \sum_{i=1}^2 0.25\alpha_i\sigma_i^2 e^{-r^2/\sigma_i^2}$,

solve the Poisson equation (I.3).

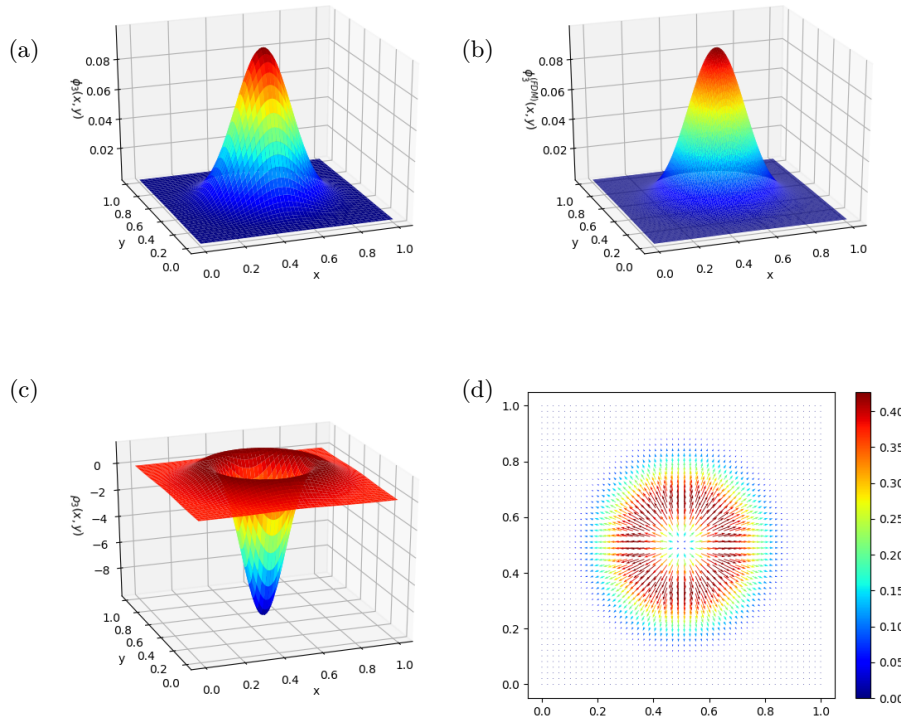


Figure 8: Example 3: Electrostatic potential computed (a) analytically and (b) by using the finite difference method (100×100 grid) ; (c) source of the potential and (d) force deriving from the potential computed by using FDM (50×50 grid).

-
- [1] P. Hohenberg and W. Kohn, "Inhomogeneous Electron Gas", Phys. Rev. **136**, B864 (1964).
 - [2] W. Kohn and L. Sham, "Self-consistent equations including exchange and correlation effects", Phys. Rev. **14**, A1133 (1965).
 - [3] J. Kohanoff, "Electronic Structure Calculations for Solids and Molecules", Cambridge University Press, 2006.
 - [4] A. D. Becke and R. M. Dickson, "Numerical solution of Poisson's equation in polyatomic molecules", J. Chem. Phys. **89**, 2993 (1988).
 - [5] J. D. Jackson, "Classical Electrodynamics", John Wiley and Sons, Inc., New York, third edition, 1999.
 - [6] S.D. Poisson, "Bulletin de la société philomatique. 1813"; D.H. Arnold. In Siméon Denis Poisson et la science de son temps. Paris. 1981.
 - [7] R. W. Hockney and J. W. Eastwood, "Computer Simulation Using Particles" (McGraw-Hill, New York, 1981).
 - [8] C. R. Anderson, J. Comput. Phys. **62**, 111 (1986).
 - [9] H. M. P. Couchman, Astrophys. J. Lett. **368**, 23 (1990).
 - [10] A. Almgren, T. Buttke, P. Colella, ibid. **113**, 177 (1994).
 - [11] A. Brandt, A. A. Lubrecht, J. Comput. Phys. **90**, 348 (1990).
 - [12] A. Brandt, Comput. Phys. Commun. **65**, 24 (1991).
 - [13] G. Beylkin, R. Coifman, V. Rokhlin, Commun. Pure Appl. Math. **44**, 141 (1991).
 - [14] A. W. Appel, SIAM J. Sci. Stat. Comput. **6**, 85 (1985).
 - [15] J. Barnes and P. Hut, Nature **324**, 446 (1986).
 - [16] L. Van Dommelen and E. A. Rundensteiner, J. Comput. Phys. **83**, 126 (1989).
 - [17] A. M. Odlyzko and A. Schönhage, Am. Math. Soc. Trans. **309**, 797 (1988).
 - [18] V. Rokhlin, J. Comput. Phys. **60**, 187 (1985).
 - [19] L. Greengard and V. Rokhlin, "A Fast Algorithm for Particle Simulations", J. Comp. Phys. **73**, 325 (1987).
 - [20] L. Greengard, "The Rapid Evaluation of Potential Fields in Particle Systems" (MIT Press, Cambridge, MA, 1988).
 - [21] L. Greengard, "Fast Algorithms for Classical Physics", Science (Washington), **265**, 909 (1994).
 - [22] C. A. White and M. Head-Gordon, "Derivation and efficient implementation of the fast multipole method", J. Chem. Phys.

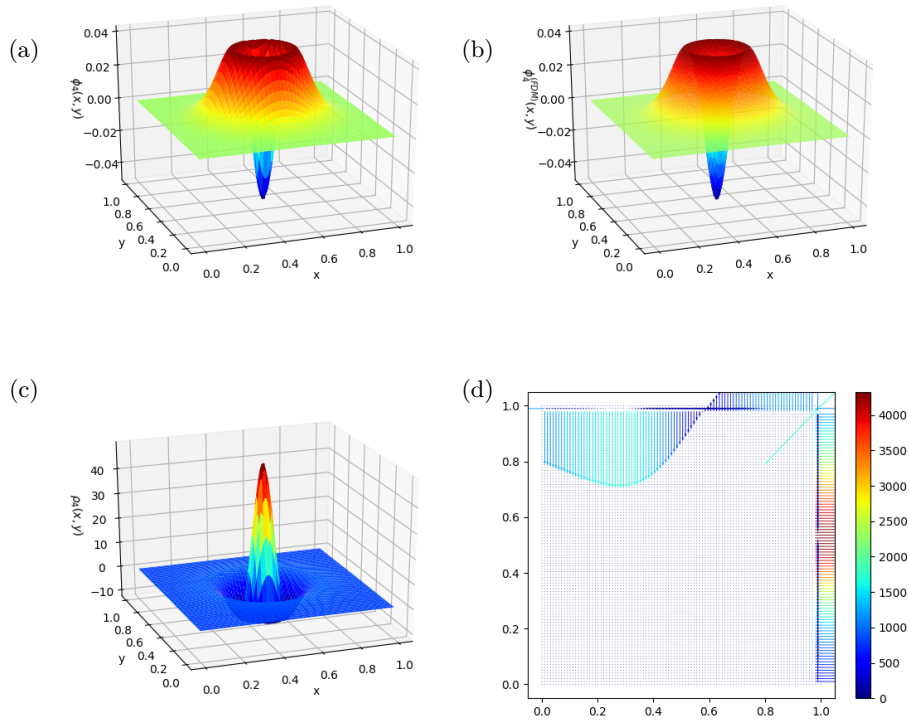


Figure 9: Example 4: Electrostatic potential computed (a) analytically and (b) by using the finite difference method (100×100 grid) ; (c) source of the potential and (d) force deriving from the potential computed by using FDM (50×50 grid).

- 101, 6593 (1994).
H. Ding, N. Karasawa, W. A. Goddard II, *ibid.* 97,4309 (1992) J. Shimada, H. Kaneko, T. Takada, *J. Comput. Chem.* 15, 28 (1994).
[23] B. L. Buzbee, G. H. Golub, and C. W. Nielson, "On Direct Methods for Solving Poisson's Equations", *SIAM J. Numer. Anal.* **7**, 627 (1970).
[24] E. Braverman, M. Israeli, A. Averbuch, L. J. Vozovoi, "A Fast 3D Poisson Solver of Arbitrary Order Accuracy", *Comput. Phys.* **144**, 109 (1998).
[25] O. Godsi, I. Bar-On, and U. Peskin, "Fast Iterative Poisson Solver for Molecular Junctions' Geometries", *J. Phys. Chem. A* **107**, 7258 (2003).
[26] R. B. Wilhelmson and J. H. Ericksen, "Direct solutions for Poisson's equation in three dimension", *J. Comp. Phys.* **25**, 319 (1977).
[27] W. M. Pickering, "On the Solution of Poisson's Equation on a Regular Hexagonal Grid Using FFT Methods", *J. Comp. Phys.* **64**, 320 (1986).
[28] J. R. Nagel, "Numerical Solutions to Poisson Equations Using Finite-Difference Method", *IEEE ANTennas and Propagation Magazine*, Vol. 56, No. 4, August 2014.
[29] "Computational Nanoscience," (2011), K. Varga and J. A. Driscoll, Cambridge University Press.
[30] Arfken, chap. 2

Code PIC2D.f90

```

program PIC2D
! cf tutorial_08_pde_finite_diff_2_solution.pdf in Zotero "Poisson's equation" library
double precision,parameter::pi=4.0*atan(1.0)
integer,parameter::dim=2
integer,parameter::level_approx=1
double precision::FD_coeff(-level_approx:level_approx)
type t_node
integer::idx(dim) ! i,j,k
double precision::r(dim)
logical::active
integer::n_active_neigh
integer::list_active_neigh(dim*(2*level_approx+1))
integer::coeff_active_neigh(dim*(2*level_approx+1))
integer::n_bound_neigh
integer::list_bound_neigh(dim*(2*level_approx+1))
integer::coeff_bound_neigh(dim*(2*level_approx+1))
end type t_node
integer, allocatable::ij2n(:, :)
double precision, allocatable::laplacian(:, :), source(:, :), u(:, :)

type(t_node), allocatable::mesh(:)
integer::N,i,j,nactive,idxn,nunactive,Ntot,ineigh,idx_neigh
double precision::width
double precision::fac,invhsqr,h ! step
double precision::r(dim)

integer::lda,ldb,nrhs,info
integer, allocatable::ipiv(:)
double precision::sol
character(len=4)::PHI
double precision::A1,A2,x0,y0,sig1,sig2,gradx,grady

! -----
! Varga & Driscoll, "computational nanoscience", p14
!
FD_coeff=(/1.0,-2.0,1.0/)
! FD_coeff=(/-1.0/12.0,4.0/3.0,-5.0/2.0,4.0/3.0,-1.0/12.0/)
! FD_coeff=(/1.0/90.0,-3.0/20.0,3.0/2.0,-49.0/18.0,3.0/2.0,-3.0/20.0,1.0/90.0/)
! FD_coeff=(/-1.0/560,8.0/315,-1.0/5.0,8.0/5.0,-205.0/72.0,8.0/5.0,-1.0/5.0,8.0/315.0,-1.0/560.0/)
! FD_coeff=(/1.0/3150.0,-5.0/1008.0,5.0/126.0,-5.0/21.0,5.0/3.0,-5269.0/1800.0,5.0/3.0,-5.0/21.0,5.0/126.0,-5.0/1008.0,
! FD_coeff=(/-1.0/16632.0,2.0/1925.0,-1.0/112.0,10.0/189.0,-15.0/56.0,12.0/7.0,-5369.0/1800.0,&
! 12.0/7.0,-15.0/56.0,10.0/189.0,-1.0/112.0,2.0/1925.0,-1.0/16632.0/)
do i=-level_approx,level_approx
print *, 'FD_coeff(' , i, ')=' , FD_coeff(i)
end do
!
! we consider a NxN nodes map of dimension widthxwidth
width=1.0
N=100
N=50
h=width/(N-1) ! step
Ntot=N*N ! size of the Laplacian matrix
allocate(mesh(Ntot))
allocate(u(Ntot)) ! the electrostatic potential
allocate(ij2n(N,N))
invhsqr=1.0/h**2
print *, 'h=', h, '1/h**2=', invhsqr

! -----
! first we set up the mesh. mesh is a data type which contains all informations about the nodes
! describing the system; for each node there is
! * the status of the node, i. e.,
! - either "active", i.e. a node where we're going to compute the electrostatic potential
! - or "inactive", i.e. a node belonging to the boundaries of the map; we need the value of
! electrostatic potential at these points in order to solve the linear system [L]|u>=|source>
! * the number and the list of active nodes of the current node (we need that to compute the
! Laplacian matrix [L])
! * the number and the list of the boundary nodes of the current node involved for computing the source
! to consider
!
nactive=0 ! "active" points are nodes where we're going to compute the electrostatic potential
nunactive=0 ! "inactive" points are nodes at the boundaries of the map; we need the value of
! electrostatic potential at these points in order to solve the linear system [L]|u>=|source>
do j=1,N
do i=1,N
r(1)=(i-1)*h ! x
r(2)=(j-1)*h ! y
if((i.gt.level_approx).and.(i.le.(N-level_approx)).and.(j.gt.level_approx).and.(j.le.(N-level_approx))) then
nactive=nactive+1
idxn=nactive
mesh(idxn)%active=.TRUE.
else
idxn=Ntot-nunactive
nunactive=nunactive+1
mesh(idxn)%active=.FALSE.
end if
ij2n(i,j)=idxn
mesh(idxn)%idx(1)=i
mesh(idxn)%idx(2)=j
mesh(idxn)%r(1)=r(1)
mesh(idxn)%r(2)=r(2)
end do
enddo
print *, 'Ntot=', Ntot, 'nactive=', nactive, 'nunactive=', nunactive, 'nactive+nunactive=', nactive+nunactive

```



```

! for each active node, we need to know its neighbors in order to build the
! Laplacian matrix [L]
do idxn=1,nactive
  mesh(idxn)%n_active_neigh=0
  mesh(idxn)%list_active_neigh=0
  mesh(idxn)%n_bound_neigh=0
  mesh(idxn)%list_bound_neigh=0
  i=mesh(idxn)%idx(1)
  j=mesh(idxn)%idx(2)
  do ineigh=-level_approx,level_approx
    ! first we check that the index of the neighbors exists
    if(((i+ineigh).ge.1).and.((i+ineigh).le.N)) then
      ! then we test the state (active or unactive) of the neighbor if it exists
      idx_neigh=ij2n(i+ineigh,j)
      if(mesh(idx_neigh)%active) then
        mesh(idxn)%n_active_neigh=mesh(idxn)%n_active_neigh+1
        mesh(idxn)%list_active_neigh(mesh(idxn)%n_active_neigh)=idx_neigh
        mesh(idxn)%coeff_active_neigh(mesh(idxn)%n_active_neigh)=FD_coeff(ineigh)
      else
        mesh(idxn)%n_bound_neigh=mesh(idxn)%n_bound_neigh+1
        mesh(idxn)%list_bound_neigh(mesh(idxn)%n_bound_neigh)=idx_neigh
        mesh(idxn)%coeff_bound_neigh(mesh(idxn)%n_bound_neigh)=FD_coeff(ineigh)
      end if
    end if
    ! first we check that the index of the neighbors exists
    if(((j+ineigh).ge.1).and.((j+ineigh).le.N)) then
      ! then we test the state (active or unactive) of the neighbor if it exists
      idx_neigh=ij2n(i,j+ineigh)
      if(mesh(idx_neigh)%active) then
        mesh(idxn)%n_active_neigh=mesh(idxn)%n_active_neigh+1
        mesh(idxn)%list_active_neigh(mesh(idxn)%n_active_neigh)=idx_neigh
        mesh(idxn)%coeff_active_neigh(mesh(idxn)%n_active_neigh)=FD_coeff(ineigh)
      else
        mesh(idxn)%n_bound_neigh=mesh(idxn)%n_bound_neigh+1
        mesh(idxn)%list_bound_neigh(mesh(idxn)%n_bound_neigh)=idx_neigh
        mesh(idxn)%coeff_bound_neigh(mesh(idxn)%n_bound_neigh)=FD_coeff(ineigh)
      end if
    end if
  end do
end do
! -----
! Building of laplacian matrix
! -----
allocate(laplacian(nactive,nactive))
laplacian=0.0
do i=1,nactive
  do j=1,mesh(i)%n_active_neigh
    ineigh=mesh(i)%list_active_neigh(j)
    laplacian(i,ineigh)=laplacian(i,ineigh)+mesh(i)%coeff_active_neigh(j)*invhsqr
  end do
end do
! For printing the laplacian matrix on screen. The matrix is symetric.
!   do i=1,nactive
!     do j=1,nactive
!       write(*,'(F6.2)',advance='no') laplacian(i,j)
!     end do
!     write(*,*)
!   end do
! -----
! !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
! SETTING THE SOURCE
! -----
! building of the source vector |source>
! -----
PHI="phi4"
A1=0.1
A2=0.15
x0=0.5
y0=0.5
sig1=0.2
sig2=0.1

allocate(source(nactive))
source=0.0
do i=1,nactive
  select case (PHI)
    case ("phi1")
      source(i)=mesh(i)%r(1)**2+mesh(i)%r(2)**2      ! source 1
    case ("phi2")
      source(i)=-20.0*cos(3*pi*mesh(i)%r(1))*sin(2*pi*mesh(i)%r(2))      ! source 2
    case ("phi3")
      source(i)=(4*A1/sig1**4)*exp(-(mesh(i)%r(1)-x0)**2+(mesh(i)%r(2)-y0)**2)/sig1**2*&
        ((mesh(i)%r(1)-x0)**2+(mesh(i)%r(2)-y0)**2-sig1**2)
    case ("phi4")
      source(i)=(4*A1/sig1**4)*exp(-(mesh(i)%r(1)-x0)**2+(mesh(i)%r(2)-y0)**2)/sig1**2*&
        ((mesh(i)%r(1)-x0)**2+(mesh(i)%r(2)-y0)**2-sig1**2)-&
        (4*A2/sig2**4)*exp(-(mesh(i)%r(1)-x0)**2+(mesh(i)%r(2)-y0)**2)/sig2**2*&

```



```

deallocate(source)
deallocate(ipiv)

```

```

contains
end program PIC2D

```

Makefile

```

INC=-I /usr/lib/x86_64-linux-gnu/openmpi/include
FC=gfortran
FCPARA=mpif90
FCOPT= -fimplicit-none -finit-local-zero -fopenmp -finit-local-zero
EXEC=PIC
all:
    $(FC) $(FCOPT) PIC2D.f90 -o $(EXEC) -lblas -llapack -lfftw3

```

Code PIC2D.py

```
#!/usr/bin/python
# Ref.: A. Devert, "Matplotlib Plotting Cookbook"
import numpy
from matplotlib import pyplot as plt
import matplotlib.cm as cm
# -----
def read_data(filename,colx,coly,colz):
    colx=colx-1
    coly=coly-1
    colz=colz-1
    F=open(filename,'r')
    data=F.read().split('\n')
    x=numpy.array([])
    y=numpy.array([])
    z=numpy.array([])
    nrow=0
    for row in data:
        if not '#' in row and len(row)>0:
            x=numpy.append(x,int(row.split()[colx]))
            y=numpy.append(y,int(row.split()[coly]))
            z=numpy.append(z,float(row.split()[colz]))
        nrow=nrow+1
    F.close()
    return x,y,z,nrow
# -----
def read_data_float(filename,colx,coly,colz):
    colx=colx-1
    coly=coly-1
    colz=colz-1
    F=open(filename,'r')
    data=F.read().split('\n')
    x=numpy.array([])
    y=numpy.array([])
    z=numpy.array([])
    nrow=0
    for row in data:
        if not '#' in row and len(row)>0:
            x=numpy.append(x,float(row.split()[colx]))
            y=numpy.append(y,float(row.split()[coly]))
            z=numpy.append(z,float(row.split()[colz]))
        nrow=nrow+1
    F.close()
    return x,y,z,nrow
# -----
def view():
    filename='ElectrostaticPotential.dat'
    colx=1
    coly=2
    colz=3
    x,y,z,Nsqr=read_data(filename,colx,coly,colz)
    print(min(z),max(z))
    N=int(numpy.sqrt(Nsqr))
    print(Nsqr,N)
    Z = numpy.empty([N,N])
    for i in range(Nsqr):
        Z[int(x[i]-1) , int(y[i]-1)]=z[i]

    plt.imshow(Z,
               cmap = cm.binary,
               interpolation = 'bicubic',
               extent=(min(x), max(x), min(y), max(y)))
    cb = plt.colorbar(orientation='horizontal', shrink=.75)
    cb.set_label('Electrostatic potential')
    plt.show()
# -----
def plot_phi1_FDM_view3D(elevation,azimuth):
    from mpl_toolkits.mplot3d import Axes3D
    filename='ElectrostaticPotential.dat'
    colx=3
    coly=4
    colz=5
    x,y,z,Nsqr=read_data_float(filename,colx,coly,colz)
    print(min(z),max(z))

    fig = plt.figure()
    ax = fig.gca(projection = '3d')
    ax.view_init(elev=elevation,azim=azimuth)
    ax.set_zlim3d(0, 25)
    ax.plot_trisurf(x,y,z,cmap=plt.cm.jet)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('$\phi_1^{(FDM)}(x,y)$')
    plt.savefig('phi1_FDM.png')
    plt.show()
# -----
def plot_phi2_FDM_view3D(elevation,azimuth):
    from mpl_toolkits.mplot3d import Axes3D
    filename='ElectrostaticPotential.dat'
    colx=3
    coly=4
    colz=5
    x,y,z,Nsqr=read_data_float(filename,colx,coly,colz)
    print(min(z),max(z))

    fig = plt.figure()
    ax = fig.gca(projection = '3d')
```

```

ax.view_init(elev=elevation,azim=azimuth)
ax.set_zlim3d(0, 1.5)
ax.plot_trisurf(x,y,z,cmap=plt.cm.jet)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('$\\phi_2^{(FDM)}(x,y)$')
plt.savefig('phi2_FDM.png')
plt.show()
# -----
def plot_phi3_FDM_view3D(elevation,azimuth):
    from mpl_toolkits.mplot3d import Axes3D
    filename='ElectrostaticPotential.dat'
    colx=3
    coly=4
    colz=5
    x,y,z,Nsq=read_data_float(filename,colx,coly,colz)
    print(min(z),max(z))

    fig = plt.figure()
    ax = fig.gca(projection = '3d')
    ax.view_init(elev=elevation,azim=azimuth)
    ax.set_zlim3d(0, 1.5)
#     ax.plot_trisurf(x,y,z,cmap=plt.cm.jet)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('$\\phi_3^{(FDM)}(x,y)$')
    plt.savefig('phi3_FDM.png')
    plt.show()
# -----
def plot_phi4_FDM_view3D(elevation,azimuth):
    from mpl_toolkits.mplot3d import Axes3D
    filename='ElectrostaticPotential.dat'
    colx=3
    coly=4
    colz=5
    x,y,z,Nsq=read_data_float(filename,colx,coly,colz)
    print(min(z),max(z))

    fig = plt.figure()
    ax = fig.gca(projection = '3d')
    ax.view_init(elev=elevation,azim=azimuth)
#     ax.set_zlim3d(0, 1.5)
    ax.plot_trisurf(x,y,z,cmap=plt.cm.jet)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('$\\phi_4^{(FDM)}(x,y)$')
    plt.savefig('phi4_FDM.png')
    plt.show()
# -----
def plot_gradphi3_FDM_view3D(elevation,azimuth):
    from mpl_toolkits.mplot3d import Axes3D
    filename='ElectrostaticPotential.dat'
    colx=3
    coly=4
    colz=5
    x,y,z,Nsq=read_data_float(filename,colx,coly,colz)
    colz=6
    x,y,u,Nsq=read_data_float(filename,colx,coly,colz)
    colz=7
    x,y,v,Nsq=read_data_float(filename,colx,coly,colz)
    M = numpy.sqrt(u*u+v*v) # magnitude
    #     print(min(z),max(z))
    qq=plt.quiver(x, y, u, v, M,cmap=plt.cm.jet,zorder=1)
#     plt.streamplot(x, y, u, v, color='k')
    plt.axes().set_aspect('equal')
    plt.colorbar(qq, cmap=plt.cm.jet)
    plt.savefig('gradphi3_FDM.png')
    plt.show()
# -----
def plot_gradphi4_FDM_view3D(elevation,azimuth):
    from mpl_toolkits.mplot3d import Axes3D
    filename='ElectrostaticPotential.dat'
    colx=3
    coly=4
    colz=5
    x,y,z,Nsq=read_data_float(filename,colx,coly,colz)
    colz=6
    x,y,u,Nsq=read_data_float(filename,colx,coly,colz)
    colz=7
    x,y,v,Nsq=read_data_float(filename,colx,coly,colz)
    M = numpy.sqrt(u*u+v*v) # magnitude
    #     print(min(z),max(z))
    qq=plt.quiver(x, y, u, v, M,cmap=plt.cm.jet,zorder=1)
#     plt.streamplot(x, y, u, v, color='k')
    plt.axes().set_aspect('equal')
    plt.colorbar(qq, cmap=plt.cm.jet)
    plt.savefig('gradphi4_FDM.png')
    plt.show()
# -----
def plot_gradphi2_FDM_view3D(elevation,azimuth):
    from mpl_toolkits.mplot3d import Axes3D
    filename='ElectrostaticPotential.dat'
    colx=3
    coly=4
    colz=5
    x,y,z,Nsq=read_data_float(filename,colx,coly,colz)
    colz=6

```

```

x,y,u,Nsqr=read_data_float(filename,colx,coly,colz)
colz=7
x,y,v,Nsqr=read_data_float(filename,colx,coly,colz)
M = numpy.sqrt(u*u+v*v) # magnitude
# print(min(z),max(z))
qq=plt.quiver(x, y, u, v, M,cmap=plt.cm.jet,zorder=1)
# plt.streamplot(x, y, u, v, color='k')
plt.axes().set_aspect('equal')
plt.colorbar(qq, cmap=plt.cm.jet)
plt.savefig('gradphi2_FDM.png')
plt.show()
# -----
def plot_source1_view3D(elevation,azimuth):
    from mpl_toolkits.mplot3d import Axes3D

    x = numpy.linspace(0, 1, 100)
    y = numpy.linspace(0, 1, 100)
    X, Y = numpy.meshgrid(x, y)
    Z = X ** 2 + Y ** 2
    fig = plt.figure()
    ax = fig.gca(projection = '3d')
    ax.set_zlim3d(0, 2.5)
    ax.view_init(elev=elevation,azim=azimuth)
    ax.plot_surface(X, Y, Z, color = 'w',cmap=cm.jet)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('$\\rho_1(x,y)$')
    plt.savefig('rho1.png')
    plt.show()
# -----
def plot_source2_view3D(elevation,azimuth):
    from mpl_toolkits.mplot3d import Axes3D

    x = numpy.linspace(0, 1, 100)
    y = numpy.linspace(0, 1, 100)
    X, Y = numpy.meshgrid(x, y)
    Z = -20*numpy.cos(3*numpy.pi*X)*numpy.sin(2*numpy.pi*Y)
    fig = plt.figure()
    ax = fig.gca(projection = '3d')
    ax.set_zlim3d(-25,25)
    ax.view_init(elev=elevation,azim=azimuth)
    ax.plot_surface(X, Y, Z, color = 'w',cmap=cm.jet)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('$\\rho_2(x,y)$')
    plt.savefig('rho2.png')
    plt.show()
# -----
def plot_phi1_analytical_view3D(elevation,azimuth):
    from mpl_toolkits.mplot3d import Axes3D

    x = numpy.linspace(0, 1, 100)
    y = numpy.linspace(0, 1, 100)
    X, Y = numpy.meshgrid(x, y)
    Z = numpy.exp(numpy.pi*X)*numpy.sin(numpy.pi*Y)+0.5*(X*Y)**2
    fig = plt.figure()
    ax = fig.gca(projection = '3d')
    ax.set_zlim3d(0, 25)
    ax.view_init(elev=elevation,azim=azimuth)
    ax.plot_surface(X, Y, Z, color = 'w',cmap=cm.jet)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('$\\phi_1^{(ana)}(x,y)$')
    plt.savefig('phi1.png')
    plt.show()
# -----
def plot_rho3_view3D(elevation,azimuth,A,x0,y0,sig):
    from mpl_toolkits.mplot3d import Axes3D

    x = numpy.linspace(0, 1, 100)
    y = numpy.linspace(0, 1, 100)
    X, Y = numpy.meshgrid(x, y)
    Z = A*numpy.exp(-((X-x0)**2+(Y-y0)**2)/sig**2)*(4.0/sig**4)*((X-x0)**2+(Y-y0)**2-sig**2)
    fig = plt.figure()
    ax = fig.gca(projection = '3d')
    ax.set_zlim3d(0, 25)
    # ax.view_init(elev=elevation,azim=azimuth)
    ax.plot_surface(X, Y, Z, color = 'w',cmap=cm.jet)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('$\\rho_3(x,y)$')
    plt.savefig('rho3.png')
    plt.show()
# -----
def plot_rho4_view3D(elevation,azimuth,A1,sig1,A2,sig2,x0,y0):
    from mpl_toolkits.mplot3d import Axes3D

    x = numpy.linspace(0, 1, 100)
    y = numpy.linspace(0, 1, 100)
    X, Y = numpy.meshgrid(x, y)
    Z = A1*numpy.exp(-((X-x0)**2+(Y-y0)**2)/sig1**2)*(4.0/sig1**4)*((X-x0)**2+(Y-y0)**2-sig1**2)-A2*numpy.exp(-((X-x0)**2+(Y-y0)**2)/sig2**2)*(4.0/sig2**4)*((X-x0)**2+(Y-y0)**2-sig2**2)
    fig = plt.figure()
    ax = fig.gca(projection = '3d')

```

```

# ax.set_zlim3d(0, 25)
ax.view_init(elev=elevation,azim=azimuth)
ax.plot_surface(X, Y, Z, color = 'w',cmap=cm.jet)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('$\\rho_4(x,y)$')
plt.savefig('rho4.png')
plt.show()
# -----
def plot_phi3_analytical_view3D(elevation,azimuth,A,x0,y0,sig):
    from mpl_toolkits.mplot3d import Axes3D
    x = numpy.linspace(0, 1, 100)
    y = numpy.linspace(0, 1, 100)
    X, Y = numpy.meshgrid(x, y)
    Z = A*numpy.exp(-((X-x0)**2+(Y-y0)**2)/sig**2)
    fig = plt.figure()
    ax = fig.gca(projection = '3d')
    # ax.set_zlim3d(0, 25)
    ax.view_init(elev=elevation,azim=azimuth)
    ax.plot_surface(X, Y, Z, color = 'w',cmap=cm.jet)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('$\\phi_3(x,y)$')
    plt.savefig('phi3.png')
    plt.show()
# -----
def plot_phi4_analytical_view3D(elevation,azimuth,A1,sig1,A2,sig2,x0,y0):
    from mpl_toolkits.mplot3d import Axes3D
    x = numpy.linspace(0, 1, 100)
    y = numpy.linspace(0, 1, 100)
    X, Y = numpy.meshgrid(x, y)
    Z = A1*numpy.exp(-((X-x0)**2+(Y-y0)**2)/sig1**2)-A2*numpy.exp(-((X-x0)**2+(Y-y0)**2)/sig2**2)
    fig = plt.figure()
    ax = fig.gca(projection = '3d')
    # ax.set_zlim3d(0, 25)
    ax.view_init(elev=elevation,azim=azimuth)
    ax.plot_surface(X, Y, Z, color = 'w',cmap=cm.jet)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('$\\phi_4(x,y)$')
    plt.savefig('phi4.png')
    plt.show()

if __name__ == '__main__':
# elevation=22.0
# azimuth=164.0
# plot_source1_view3D(elevation,azimuth)
# plot_phi1_analytical_view3D(elevation,azimuth)
# plot_phi1_FDM_view3D(elevation,azimuth)
# elevation=24
# azimuth=-110
# plot_source2_view3D(elevation,azimuth)
# plot_phi1_analytical_view3D(elevation,azimuth)
# plot_phi2_FDM_view3D(elevation,azimuth)
# plot_gradphi2_FDM_view3D(elevation,azimuth)
# elevation=24
# azimuth=-110
# A=0.1
# x0=0.5
# y0=0.5
# sig=0.2
# plot_rho3_view3D(elevation,azimuth,A,x0,y0,sig)
# plot_phi3_analytical_view3D(elevation,azimuth,A,x0,y0,sig)
# plot_phi3_FDM_view3D(elevation,azimuth)
# plot_gradphi3_FDM_view3D(elevation,azimuth)
elevation=24
azimuth=-110
A1=0.1
A2=0.15
x0=0.5
y0=0.5
sig1=0.2
sig2=0.1
plot_rho4_view3D(elevation,azimuth,A1,sig1,A2,sig2,x0,y0)
plot_phi4_analytical_view3D(elevation,azimuth,A1,sig1,A2,sig2,x0,y0)
plot_phi4_FDM_view3D(elevation,azimuth)
plot_gradphi4_FDM_view3D(elevation,azimuth)

```